CSS 3

Où écrit-on le CSS ?

On peut écrire du code en langage CSS à trois endroits différents :

- dans un fichier .css(*méthode la plus recommandée*) ;
- dans l'en-tête <head>du fichier HTML ;
- directement dans les balises du fichier HTML *via* un attribut style(*méthode la moins recommandée*).

Dans un fichier `.css` (recommandé)

On va partir du fichier HTML suivant :

```
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8" />
        <link rel="stylesheet" href="style.css" />
        <title>Premiers tests du CSS</title>
    </head>

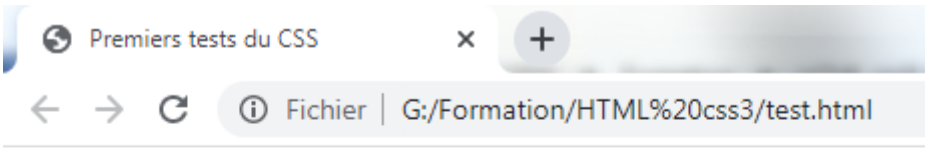
    <body>
        <h1>Mon super site</h1>

        <p>Bonjour et bienvenue sur mon site !</p>
        <p>Pour le moment, mon site est un peu <em>vide</em>. Patientez
        encore un peu !</p>
    </body>
</html>
```

Fichier style.css

```
p
{
    color: blue;

}
```



Mon super site

Bonjour et bienvenue sur mon site !

Pour le moment, mon site est un peu *vide*. Patientez encore un peu !

Dans l'en-tête <head> du fichier HTML

```
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8" />
        <style>
            p
            {
                color: blue;
            }
        </style>
        <title>Premiers tests du CSS</title>
    </head>
    <body>
        <h1>Mon super site</h1>

        <p>Bonjour et bienvenue sur mon site !</p>
        <p>Pour le moment, mon site est un peu <em>vide</em>. Patientezencore
un peu !</p>
    </body>
</html>
```

Directement dans les balises (non recommandé)

```
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8" />
        <title>Premiers tests du CSS</title>
    </head>

    <body>
        <h1>Mon super site</h1>
        <p style="color: blue;">Bonjour et bienvenue sur mon site !</p>
        <p>Pour le moment, mon site est un peu <em>vide</em>. Patientezencore
un peu !</p>
    </body>
</html>
```

Appliquer un style : class et id

```
<h1 class=""> </h1>
<p class=""> </p>
<img id="" />

.introduction
{
    color: blue;
}

 #logo
 {
     /* Indiquez les propriétés CSS ici */
 }
```

Les balises universelles

- `<span> </span>` : c'est une balise de type **inline**, c'est-à-dire une balise que l'on place au sein d'un paragraphe de texte, pour sélectionner certains mots uniquement. Les balises `<strong>` et `<em>` sont de la même famille. Cette balise s'utilise donc au milieu d'un paragraphe et c'est celle dont nous allons nous servir pour colorer « bienvenue ».
- `<div> </div>` : c'est une balise de type **block**, qui entoure un bloc de texte. Les balises `<p>`, `<h1>`, etc. sont de la même famille. Ces balises ont quelque chose en commun : elles créent un nouveau « bloc
» dans la page et provoquent donc obligatoirement un retour à la ligne.

Les sélecteurs avancés

*** : sélecteur universel**

```
*
{

}
```

A B : une balise contenue dans une autre

```
h3 em
{

}
```

A + B : une balise qui en suit une autre

```
h3 + p
{

}
```

A[attribut] : une balise qui possède un attribut

```
a[title]
{

}
```

A[attribut="Valeur"] : une balise, un attribut et une valeur exacte

```
a[title="Cliquez
ici"]
{

}
```

A[attribut*="Valeur"] : une balise, un attribut et une valeur

```
a[title*="ici"]
{

}
```

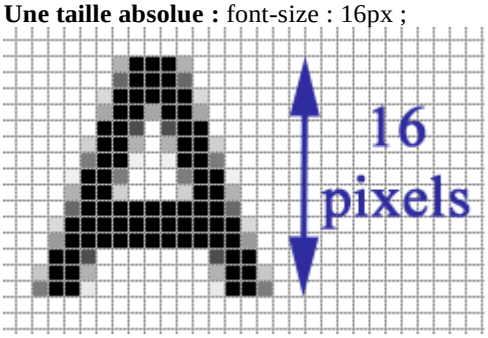
Formatage du texte

La taille

On utilise la propriété CSS `font-size`.

- Indiquer une **taille absolue** : en pixels, en centimètres ou millimètres. Cette méthode est très précise mais il est conseillé de ne l'utiliser que si c'est absolument nécessaire, car on risque d'indiquer une taille trop petite pour certains lecteurs.
- Indiquer une **taille relative** : en pourcentage, « em » ou « ex », cette technique a l'avantage d'être plus souple. Elle s'adapte plus facilement aux préférences de taille des visiteurs.

Une taille absolue : font-size : 16px ;



Une valeur relative

- `xx-small` : minuscule ;
- `x-small` : très petit ;
- `small` : petit ;
- `medium` : moyen ;
- `large` : grand ;
- `x-large` : très grand ;
- `xx-large` : euh… gigantesque.

```
p
{
font-size: 0.8em;
}
h1
{
font-size: 1.3em;
}
```

La police

La propriété CSS qui permet d'indiquer la police à utiliser est `font-family`.

```
balise
{
    font-family: police;
}

balise
```

```
{
    font-family: police1, police2, police3, police4;
}

@font-face {
    font-family: 'MaSuperPolice'; src:
    url('MaSuperPolice.eot');
}
```

Italique, gras, souligné…

Mettre en italique

En CSS, pour mettre en italique, on utilise `font-style` qui peut prendre trois valeurs :

- `italic` : le texte sera mis en italique.
- `oblique` : le texte sera passé en oblique (les lettres sont penchées, le résultat est légèrement différent de l'italique proprement dit).
- Normal : le texte sera normal (par défaut)

Mettre en gras

La propriété CSS pour mettre en gras est `font-weight` et prend les valeurs suivantes :

- `bold` : le texte sera en gras ;
- normal : le texte sera écrit normalement (par défaut).

Soulignement et autres décorations

La propriété CSS associée porte bien son nom : `text-decoration`. Elle permet, entre autres, de souligner le texte, mais pas seulement. Voici les différentes valeurs qu'elle peut prendre :

- `underline` : souligné.
- `line-through` : barré.
- `overline` : ligne au-dessus.
- `blink` : clignotant. Ne fonctionne pas sur tous les navigateurs (Internet Explorer et Google Chrome, notamment).
- None : normal (par defaut)

L'alignement

Le langage CSS nous permet de faire tous les alignements connus : à gauche, centré, à droite et justifié.

C'est tout simple. On utilise la propriété `text-align` et on indique l'alignement désiré :

- `left` : le texte sera aligné à gauche (c'est le réglage par défaut).
- `center` : le texte sera centré.
- `right` : le texte sera aligné à droite.
- `justify` : le texte sera « justifié ».

Les flottants

Le CSS nous permet de faire flotter un élément autour du texte. On dit aussi qu'on fait un « habillage ».

`float` (« flottant » en anglais). Cette propriété peut prendre deux valeurs très simples :

- `left` : l'élément flottera à gauche.
- `right` : l'élément flottera… à droite ! Oui, bravo.

L'utilisation des flottants est très simple :

Vous appliquez un `float` à une balise. Puis vous continuez à écrire du texte à la suite normalement.

Faire flotter une image

Voici le code HTML que nous devons taper dans un premier temps :

```
<p><img src="flash.gif" class="imageflottante" alt="Image flottante" /> Ceci est un texte normal de paragraphe, écrit à la suite de l'image et qui l'habillera car l'image est flottante.</p>
```

Voici le seul bout de code CSS

```
.imageflottante
{
    float: left;
}
```



Ce texte entoure l'image. Bla bla bla bla bla bla bla bla

Texte écrit en-dessous

la propriété `clear`, qui peut prendre ces trois valeurs :

- `left` : le texte se poursuit en-dessous après un `float: left;`
- `right` : le texte se poursuit en-dessous après un `float: right;`
- `both` : le texte se poursuit en-dessous, que ce soit après un `float: left;` ou après un `float: right;`.

Couleur de fond

Pour indiquer une couleur de fond, on utilise la propriété CSS `background-color`.

Images de fond

La propriété permettant d'indiquer une image de fond est `background-image`. Comme valeur, on doit renseigner `url("nom_de_l_image.png")`. Par exemple :

```
body
{
    background-image: url("neige.png");
}
```

Options disponibles pour l'image de fond

On peut compléter la propriété `background-image` que nous venons de voir par plusieurs autres propriétés qui permettent de changer le comportement de l'image de fond.

`background-attachment` : fixer le fond

La propriété CSS `background-attachment` permet de « fixer » le fond. L'effet obtenu est intéressant car on voit alors le texte « glisser » par-dessus le fond. Deux valeurs sont disponibles :

- `fixed` : l'image de fond reste fixe ;
- `scroll` : l'image de fond défile avec le texte (par défaut).

```
body
{
    background-image: url("neige.png");

    background-attachment: fixed; /* Le fond restera fixe */
```

`background-repeat` : répétition du fond

Par défaut, l'image de fond est répétée en mosaïque. Vous pouvez changer cela avec la propriété `background-repeat` :

- `no-repeat` : le fond ne sera pas répété. L'image sera donc unique sur la page.
- `repeat-x` : le fond sera répété uniquement sur la première ligne, horizontalement.
- `repeat-y` : le fond sera répété uniquement sur la première colonne, verticalement.
- `repeat` : le fond sera répété en mosaïque (par défaut).

Exemple d'utilisation :

```
body
{
    background-image: url("soleil.png");

    background-repeat: no-repeat;
```

`background-position` : position du fond

On peut indiquer où doit se trouver l'image de fond avec `background-position`. Cette propriété n'est intéressante que si elle est combinée avec `background-repeat: no-repeat;` (un fond qui ne se répète pas).

Vous devez donner à `background-position` deux valeurs en pixels pour indiquer la position du fond par rapport au coin supérieur gauche de la page (ou du paragraphe, si vous appliquez le fond à un paragraphe). Ainsi, si vous tapez :

```
background-position: 30px 50px;
```

… votre fond sera placé à 30 pixels de la gauche et à 50 pixels du haut. Il est aussi possible d'utiliser ces valeurs en anglais :

- `top` : en haut ;
- `bottom` : en bas ;
- `left` : à gauche ;
- `center` : centré ;
- `right` : à droite.

Il est possible de combiner ces mots. Par exemple, pour aligner une image en haut à droite, vous taperez :

```
background-position: top right;
```

Ainsi, si je veux afficher un soleil en image de fond (figure suivante), en un unique exemplaire (`no-repeat`), toujours visible (`fixed`) et positionné en haut à droite (`top right`), je vais écrire ceci :

```
body
{
    background-image: url("soleil.png");

    background-attachment: fixed; /* Le fond restera fixe */

    background-repeat: no-repeat; /* Le fond ne sera pas répété */

    background-position: top right; /* Le fond sera placé en haut à droite
```

Combiner les propriétés

Si vous utilisez beaucoup de propriétés en rapport avec le fond (comme c'est le cas sur ce dernier exemple), vous pouvez utiliser une sorte de « super-propriété » appelée `background` dont la valeur peut combiner plusieurs des propriétés vues précédemment : `background-image`, `background-repeat`, `background-attachment` et `background-position`.

On peut donc tout simplement écrire :

```
body
{
    background: url("soleil.png") fixed no-repeat top right;
```

C'est la première « super-propriété » que je vous montre, il y en aura d'autres. Il faut savoir que :

- L'ordre des valeurs n'a pas d'importance. Vous pouvez combiner les valeurs dans n'importe quel ordre.

Plusieurs images de fond

Depuis CSS3, il est possible de donner plusieurs images de fond à un élément. Pour cela, il suffit de séparer les déclarations par une virgule, comme ceci :

```
body
{
    background: url("soleil.png") fixed no-repeat top right,
url("neige.png") fixed;
```

La transparence

Le CSS nous permet de jouer très facilement avec les niveaux de transparence des éléments ! Pour cela, nous allons utiliser des fonctionnalités de CSS3 : la propriété `opacity` et la notation RGBa.

La propriété `opacity`

La propriété `opacity`, très simple, permet d'indiquer le niveau d'opacité (c'est l'inverse de la transparence).

- Avec une valeur de 1, l'élément sera totalement opaque : c'est le comportement par défaut.
- Avec une valeur de 0, l'élément sera totalement transparent.

Il faut donc choisir une valeur comprise entre 0 et 1. Ainsi, avec une valeur de 0.6, votre élément sera opaque à 60%… et on verra donc à travers !

Voici comment on peut l'utiliser :

```
p
{
    opacity: 0.6;
```

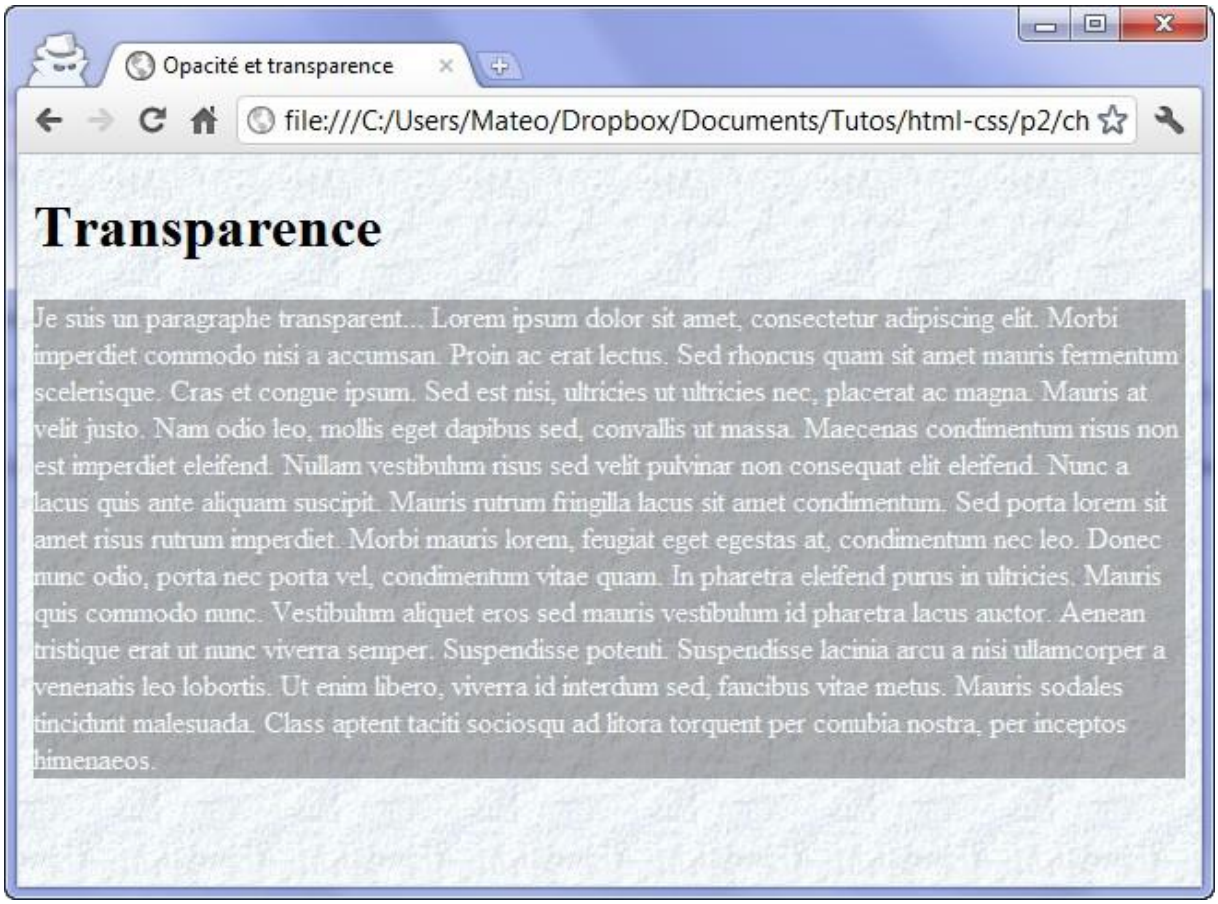
Voici un exemple qui va nous permettre d'apprécier la transparence. Vous en trouverez le rendu à la figure suivante.

```css
body
{
    background: url('neige.png');

}

p
{
    background-color: black;

    color: white;
```



Un paragraphe transparent

La notation RGBa

CSS3 nous propose une autre façon de jouer avec la transparence : la notation RGBa. Il s'agit en fait de la notation RGB que nous avons vue précédemment, mais avec un quatrième paramètre : le niveau de transparence (appelé « canal alpha »). De la même façon que précédemment, avec une valeur de 1, le fond est complètement opaque. Avec une valeur inférieure à 1, il est transparent.

```css
p
{
    background-color: rgba(255, 0, 0, 0.5); /* Fond rouge à moitié
transparent */
```

C'est aussi simple que cela. Vous pouvez obtenir exactement le même effet qu'avec `opacity` juste en jouant avec la notation RGBa, essayez !

Cette notation est connue de tous les navigateurs récents, y compris Internet Explorer (à partir de IE9). Pour les navigateurs plus anciens, il est recommandé d'indiquer la notation RGB classique en plus de RGBa.
Pour ces navigateurs, le fond ne sera alors pas transparent mais, au moins, il y aura bien une couleur d'arrière-plan.

```
p
{
    background-color: rgb(255,0,0); /* Pour les navigateurs anciens */

    background-color: rgba(255,0,0,0.5); /* Pour les navigateurs plus
récents */
```

Les bordures et les ombres

Bordures standard

nombreuses propriétés CSS vous permettent de modifier l'apparence de vos bordures : `border-width`, `border-color`, `border-style`...

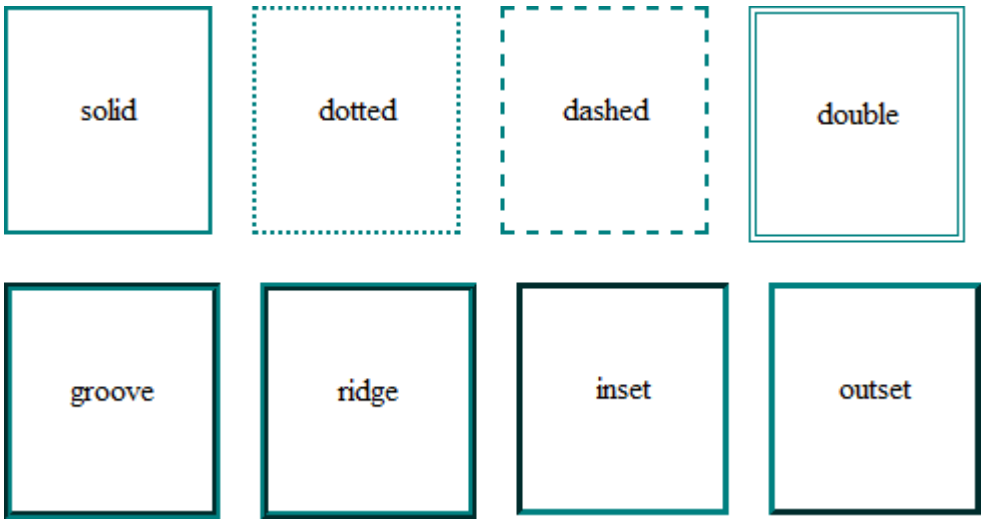
Pour `border` on peut utiliser jusqu'à trois valeurs pour modifier l'apparence de la bordure :

- **La largeur** : indiquez la largeur de votre bordure. Mettez une valeur en pixels (comme `2px`).
- **La couleur** : c'est la couleur de votre bordure. Utilisez, comme on l'a appris, soit un nom de couleur (`black`, `red`,...), soit une valeur hexadécimale (`#FF0000`), soit une valeur RGB (`rgb(198, 212, 37)`).
- **Le type de bordure** : là, vous avez le choix. Votre bordure peut être un simple trait, ou des pointillés, ou encore des tirets, etc. Voici les différentes valeurs disponibles :
    - `none` : pas de bordure (par défaut) ;
    - `solid` : un trait simple ;
    - `dotted` : pointillés ;
    - `dashed` : tirets ;
    - `double` : bordure double ;
    - `groove` : en relief ;
    - `ridge` : autre effet relief ;
    - `inset` : effet 3D global enfoncé ;
    - `outset` : effet 3D global surélevé.

Ainsi, pour avoir une bordure bleue, en tirets, épaisse de 3 pixels autour de mes titres, je vais écrire :

```
h1
{
    border: 3px blue dashed;
```

La figure suivante vous présente les différents styles de bordures que vous pouvez utiliser.



Les différents types de bordures

En haut, à droite, à gauche, en bas…

Les quatre propriétés :

- `border-top` : bordure du haut ;
- `border-bottom` : bordure du bas ;
- `border-left` : bordure de gauche ;
- `border-right` : bordure de droite.

Pour ajouter une bordure uniquement à gauche et à droite des paragraphes, on écrira donc :

```
p
{
    border-left: 2px solid black;
    border-right: 2px solid black;
```

Bordures arrondies

La propriété `border-radius` va nous permettre d'arrondir facilement les angles de n'importe quel élément. Il suffit d'indiquer la taille (« l'importance ») de l'arrondi en pixels :

```
p
{
    border-radius: 10px;
}
```

On peut aussi préciser la forme de l'arrondi pour chaque coin. Dans ce cas, indiquez quatre valeurs :

```
p
{
    border-radius: 10px 5px 10px 5px;
}
```

Les valeurs correspondent aux angles suivants dans cet ordre :

1. en haut à gauche ;
2. en haut à droite ;
3. en bas à droite ;
4. en bas à gauche.

Les ombres

Les ombres font partie des nouveautés récentes proposées par CSS3. Nous allons ici découvrir deux types d'ombres :

- les ombres des boîtes ;
- les ombres du texte.

`box-shadow` : les ombres des boîtes

La propriété `box-shadow` s'applique à tout le bloc et prend quatre valeurs dans l'ordre suivant :

1. le décalage horizontal de l'ombre ;
2. le décalage vertical de l'ombre ;
3. l'adoucissement du dégradé ;
4. la couleur de l'ombre ;

Par exemple, pour une ombre noire de 6 pixels, sans adoucissement, on écrira :

```
p
{
    box-shadow: 6px 6px 0px black;

}
```

On peut aussi rajouter une cinquième valeur facultative : `inset`. Dans ce cas, l'ombre sera placée à l'intérieur du bloc, pour donner un effet enfoncé :

```
p
{
    box-shadow: 6px 6px 6px black inset;

}
```

`text-shadow` : l'ombre du texte

Avec `text-shadow`, vous pouvez ajouter une ombre directement sur les lettres de votre texte ! Les valeurs fonctionnent exactement de la même façon que `box-shadow` : décalage, adoucissement et couleur.

```
p
{
    text-shadow: 2px 2px 4px black;
```

Création d'apparences dynamiques

Nous verrons dans ce chapitre comment changer l'apparence :

- au survol ;
- lors du clic ;
- lors du focus (élément sélectionné) ;
- lorsqu'un lien a été consulté.

Au survol

`:hover`.

```
a:hover
{

}
```

```
a /* Liens par défaut (non survolés) */
{
   text-decoration: none;

   color: red;

   font-style: italic;

}

a:hover /* Apparence au survol des liens */
{
   text-decoration: underline;

   color: green;

}
```

On a défini ici deux versions des styles pour les liens :

- pour les liens par défaut (non survolés) ;
- pour les liens au survol.

Au clic et lors de la sélection

Nous pouvons changer l'apparencedes éléments lorsque l'on clique dessus et lorsqu'ils sont sélectionnés !

`:active` : au moment du clic

Le pseudo-format `:active` permet d'appliquer un style particulier *au moment du clic*. En pratique, il n'est utilisé que sur les liens.
Le lien gardera cette apparence très peu de temps.
On peut par exemple changer la couleur de fond du lien lorsque l'on clique dessus :

```
a:active /* Quand le visiteur clique sur le lien */
{
    background-color: #FFCC66;

}
```

`:focus` : lorsque l'élément est sélectionné

Là, c'est un peu différent. Le pseudo-format `:focus` applique un style *lorsque l'élément est sélectionné*.

Lorsque le lien a déjà été consulté

Il est possible d'appliquer un style à un lien vers une page qui a déjà été vue. Par défaut, le navigateur colore le lien en un violet assez laid (de mon point de vue du moins !).

Vous pouvez changer cette apparence avec `:visited` (qui signifie « visité »).

```
a:visited /* Quand le visiteur a déjà vu la page concernée */
{
    color: #AAA; /* Appliquer une couleur grise */
}
```

Mise en page du site

Structurer sa page

Les balises structurantes de HTML5

`<header>` : l'en-tête

```
<header>
    <!-- Placez ici le contenu de l'en-tête de votre page -->
</header>
```

`<footer>` : le pied de page

À l'inverse de l'en-tête, le pied de page se trouve en général tout en bas du document. On y trouve des informations comme des liens de contact, le nom de l'auteur, les mentions légales, etc.

```
<footer>
    <!-- Placez ici le contenu du pied de page -->
</footer>
```

`<nav>` : principaux liens de navigation

La balise `<nav>` doit regrouper tous les principaux liens de navigation du site. Vous y placerez par exemple le menu principal de votre site.

Généralement, le menu est réalisé sous forme de liste à puces à l'intérieur de la balise `<nav>` :

```
<nav>
    <ul>
        <li><a  href="index.html">Accueil</a></li>
        <li><a  href="forum.html">Forum</a></li>
        <li><a  href="contact.html">Contact</a></li>
    </ul>
</nav>
```

<section> : une section de page

La balise <section> sert à regrouper des contenus en fonction de leur thématique. Elle englobe généralement une portion du contenu au centre de la page.

```
<section>
    <h1>Ma section de page</h1>
    <p>Bla bla bla bla</p>
</section>
```

<aside> : informations complémentaires

La balise <aside> est conçue pour contenir des informations complémentaires au document que l'on visualise. Ces informations sont généralement placées sur le côté (bien que ce ne soit pas une obligation).

```
<aside>
    <!-- Placez ici des informations complémentaires -->
</aside>
```

Il peut y avoir plusieurs blocs <aside> dans la page.

<article> : un article indépendant

La balise <article> sert à englober une portion généralement autonome de la page. C'est une partie de la page qui pourrait ainsi être reprise sur un autre site. C'est le cas par exemple des actualités (articles de journaux ou de blogs).

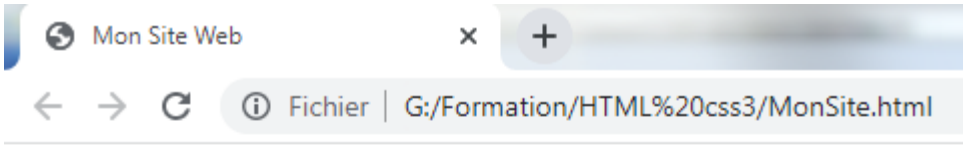
```
<article>
    <h1>Mon article</h1>
    <p>Bla bla bla bla</p>
</article>
```

Exemple concret d'utilisation des balises

```
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8" />
        <title>Mon Site Web</title>
    </head>
    <body>
        <header>
            <h1>Mon Site</h1>
            <h2>Carnets de voyage</h2>
        </header>
        <nav>
            <ul>
                <li><a href="#">Accueil</a></li>
                <li><a href="#">Blog</a></li>
                <li><a href="#">CV</a></li>
            </ul>
```

```
        </nav>
        <section>
            <aside>
                <h1>À propos de l'auteur</h1>
                <p>C'est moi ! Je suis né un 23 novembre 2005.</p>
            </aside>
            <article>
                <h1>Je suis un grand voyageur</h1>
                <p>Bla bla bla bla (texte de l'article)</p>
            </article>
        </section>

        <footer>
            <p>Copyright Moi - Tous droits réservés<br />
            <a href="#">Me contacter !</a></p>
        </footer>
    </body>
</html>
```



Mon Site

Carnets de voyage

- Accueil
- Blog
- CV

A propos de l'auteur

C'est moi ! Je suis né un 23 novembre 2005.

Je suis un grand voyageur

Bla bla bla bla (texte de l'article)

Copyright Moi - Tous droits réservés
Me contacter !

En résumé

- Plusieurs balises ont été introduites avec HTML5 pour délimiter les différentes zones qui constituent la page web :
    - `<header>` : en-tête ;
    - `<footer>` : pied de page ;
    - `<nav>` : liens principaux de navigation ;
    - `<section>` : section de page ;
    - `<aside>` : informations complémentaires ;
    - `<article>` : article indépendant.
- Ces balises peuvent être imbriquées les unes dans les autres. Ainsi, une section peut avoir son propre en-tête.
- Ces balises ne s'occupent pas de la mise en page. Elles servent seulement à indiquer à l'ordinateur le sens du texte qu'elles contiennent. On pourrait très bien placer l'en-tête en bas de la page si on le souhaite.

Les balises de type block et inline

En HTML, la plupart des balises peuvent se ranger dans l'une ou l'autre de deux catégories :

- Les balises **inline** : c'est le cas par exemple des liens `<a></a>`.
- Les balises **block** : c'est le cas par exemple des paragraphes `<p></p>`.

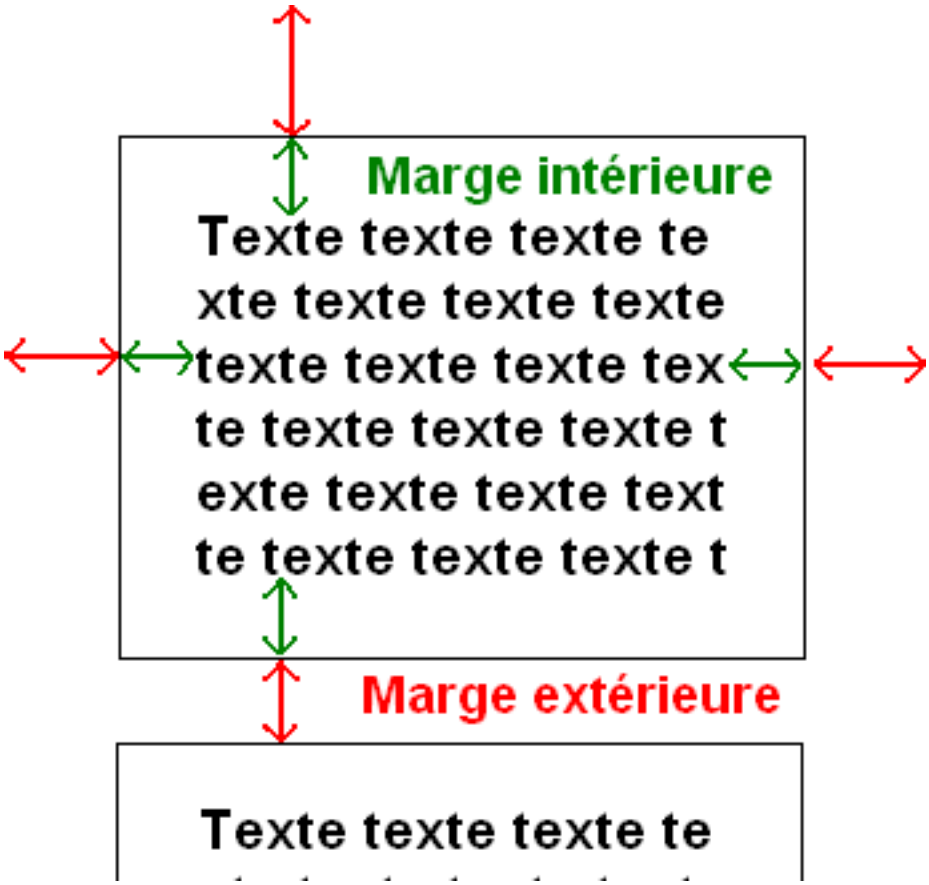
Quelques exemples

Balises block	Balises inline
<p>	
<footer>	
<h1>	<mark>
<h2>	<a>
<article>	<img />
…	…

Les marges

Il faut savoir que tous les blocs possèdent des marges. Il existe *deux types de marges* :

- les marges intérieures ;
- les marges extérieures.



Marges extérieure et intérieure

Sur ce bloc, j'ai mis une bordure pour qu'on repère mieux ses frontières.

- L'espace entre le texte et la bordure est la marge intérieure (en vert).
- L'espace entre la bordure et le bloc suivant est la marge extérieure (en rouge).

En CSS, on peut modifier la taille des marges avec les deux propriétés suivantes :

- `padding` : indique la taille de la marge intérieure. À exprimer en général en pixels (px).
- `margin` : indique la taille de la marge extérieure. Là encore, on utilise le plus souvent des pixels.

En haut, à droite, à gauche, en bas… Et on recommence !

L'idéal serait que vous reteniez les termes suivants en anglais :

- *top* : haut ;
- *bottom* : bas ;
- *left* : gauche ;
- *right* : droite.

La liste des propriétés pour `margin` et `padding`.

Voici la liste pour `margin` :

- `margin-top` : marge extérieure en haut ;
- `margin-bottom` : marge extérieure en bas ;
- `margin-left` : marge extérieure à gauche ;
- `margin-right` : marge extérieure à droite.

Et la liste pour `padding` :

- `padding-top` : marge intérieure en haut ;
- `padding-bottom` : marge intérieure en bas ;
- `padding-left` : marge intérieure à gauche ;
- `padding-right` : marge intérieure à droite.

Centrer des blocs

Il est tout à fait possible de centrer des blocs. C'est même très pratique pour réaliser un design centré quand on ne connaît pas la résolution du visiteur.

Pour centrer, il faut respecter les règles suivantes :

- donnez une largeur au bloc (avec la propriété `width`) ;
- indiquez que vous voulez des marges extérieures automatiques, comme ceci : `margin: auto;`.

```
p
{
    width: 350px; /* On a indiqué une largeur (obligatoire) */
    margin: auto; /* On peut donc demander à ce que le bloc soit centré
avec auto */
    border: 1px solid black;
    text-align: justify;
    padding: 12px;

    margin-bottom: 20px;

}
```

Quand ça dépasse…

Lorsqu'on commence à définir des dimensions précises pour nos blocs, comme on vient de le faire, il arrive qu'ils deviennent trop petits pour le texte qu'ils contiennent.

Les propriétés CSS que nous allons voir ici ont justement été créées pour contrôler les dépassements… et décider quoi faire si jamais cela devait arriver.

`overflow` : couper un bloc

Supposons que vous ayez un long paragraphe et que vous vouliez (pour une raison qui ne regarde que vous) qu'il fasse 250 px de large et 110 px de haut. Ajoutons-lui une bordure et remplissons-le de texte… à ras-bord (figure suivante) :

```
p
{
    width: 250px;
    height: 110px;
    text-align: justify;
    border: 1px solid black;
}
```

Si vous voulez que le texte ne dépasse pas des limites du paragraphe, il va falloir utiliser la propriété `overflow`. Voici les valeurs qu'elle peut accepter :

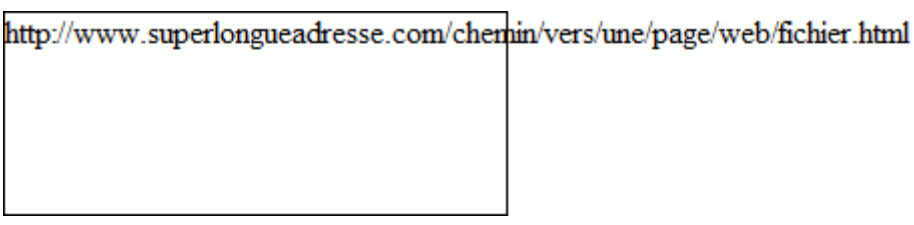
- `visible` (par défaut) : si le texte dépasse les limites de taille, il reste visible et sort volontairement du bloc.
- `hidden` : si le texte dépasse les limites, il sera tout simplement coupé. On ne pourra pas voir tout le texte.
- `scroll` : là encore, le texte sera coupé s'il dépasse les limites. Sauf que cette fois, le navigateur mettra en place des barres de défilement pour qu'on puisse lire l'ensemble du texte. C'est un peu comme un cadre à l'intérieur de la page.
- `auto` : c'est le mode « pilote automatique ». En gros, c'est le navigateur qui décide de mettre ou non des barres de défilement (il n'en mettra que si c'est nécessaire). C'est la valeur que je conseille d'utiliser le plus souvent.

Avec `overflow: hidden;` le texte est donc coupé (on ne peut pas voir la suite).

`word-wrap` : couper les textes trop larges

`word-wrap` : Cette propriété permet de forcer la césure des très longs mots (généralement des adresses un peu longues).

La figure suivante représente ce que l'on peut avoir quand on écrit une URL un peu longue dans un bloc.



Le texte déborde en largeur

L'ordinateur ne sait pas « couper » l'adresse car il n'y a ni espace, ni tiret. Il ne sait pas faire la césure.

Avec le code suivant, la césure sera forcée dès que le texte risque de dépasser (figure suivante).

```
p
{
    word-wrap: break-word;

}
```

Quelques autres techniques de mise en page

Aujourd'hui, Flexbox est l'outil de mise en page que je recommande... mais vous devriez toujours connaître les autres techniques de mise en page que je vais vous présenter dans ce chapitre. Plus anciennes, elles ont l'avantage d'être reconnues par les vieilles versions des navigateurs. Enfin, si vous êtes amenés à gérer un "vieux" code, vous aurez besoin de connaître ces techniques de positionnement !

Le positionnement flottant

Vous vous souvenez de la propriété `float` ? Nous l'avons utilisée pour faire flotter une image autour du texte.

```
nav
{
    float: left;

    width: 150px;

    border: 1px solid black;

}
section
{
    border: 1px solid blue;

}
```



```
nav
{
    float: left;

    width: 150px;

    border: 1px solid black;

}
section
{
    margin-left: 170px;

    border: 1px solid blue;

}
```



Transformez vos éléments avec display

Il existe en CSS une propriété très puissante : `display`. Elle est capable de *transformer* n'importe quel élément de votre page d'un type vers un autre. Avec cette propriété magique, je peux par exemple imposer à mes liens (originellement de type inline) d'apparaître sous forme de blocs :

```
a
{
    display: block;

}
```

À ce moment-là, les liens vont se positionner les uns en-dessous des autres (comme des blocs normaux) et il devient possible de modifier leurs dimensions !

Voici quelques-unes des principales valeurs que peut prendre la propriété `display` en CSS (il y en a encore d'autres) :

Valeur	Exemples	Description

`inline`	`<a>`, `<em>`, `<span>`...	Eléments d'une ligne. Se placent les uns à côté des autres.
`block`	`<p>`, `<div>`, `<section>`...	Eléments en forme de blocs. Se placent les uns en-dessous des autres et peuvent être redimensionnés.
`inline-block`	`<select>`, `<input>`	Eléments positionnés les uns à côté des autres (comme les inlines) mais qui peuvent être redimensionnés (comme les blocs).
`none`	`<head>`	Eléments non affichés.

On peut donc décider de masquer complètement un élément de la page avec cette propriété. Par exemple, si je veux masquer les éléments qui ont la classe « secret », je vais écrire :

```
.secret
{
    display: none;

}
```

Peu de balises sont affichées comme cela par défaut, c'est surtout le cas des éléments de formulaire (comme `<input>`) que nous découvrirons un peu plus tard. Par contre, avec la propriété `display`, nous allons pouvoir transformer d'autres balises en `inline-block`, ce qui va nous aider à réaliser notre design.

Le positionnement inline-block

Une meilleure technique consiste à transformer vos éléments en `inline-block` avec la propriété `display`.

Quelques petits rappels sur les éléments de type inline-block :

- Ils se positionnent les uns à côté des autres (exactement ce qu'on veut pour placer notre menu et le corps de notre page !).
- On peut leur donner des dimensions précises (là encore, exactement ce qu'on veut !).

Nous allons transformer en inline-block les deux éléments que nous voulons placer côte à côte : le menu de navigation et la section du centre de la page.

```
nav
{
    display: inline-block;

    width: 150px;

    border: 1px solid black;

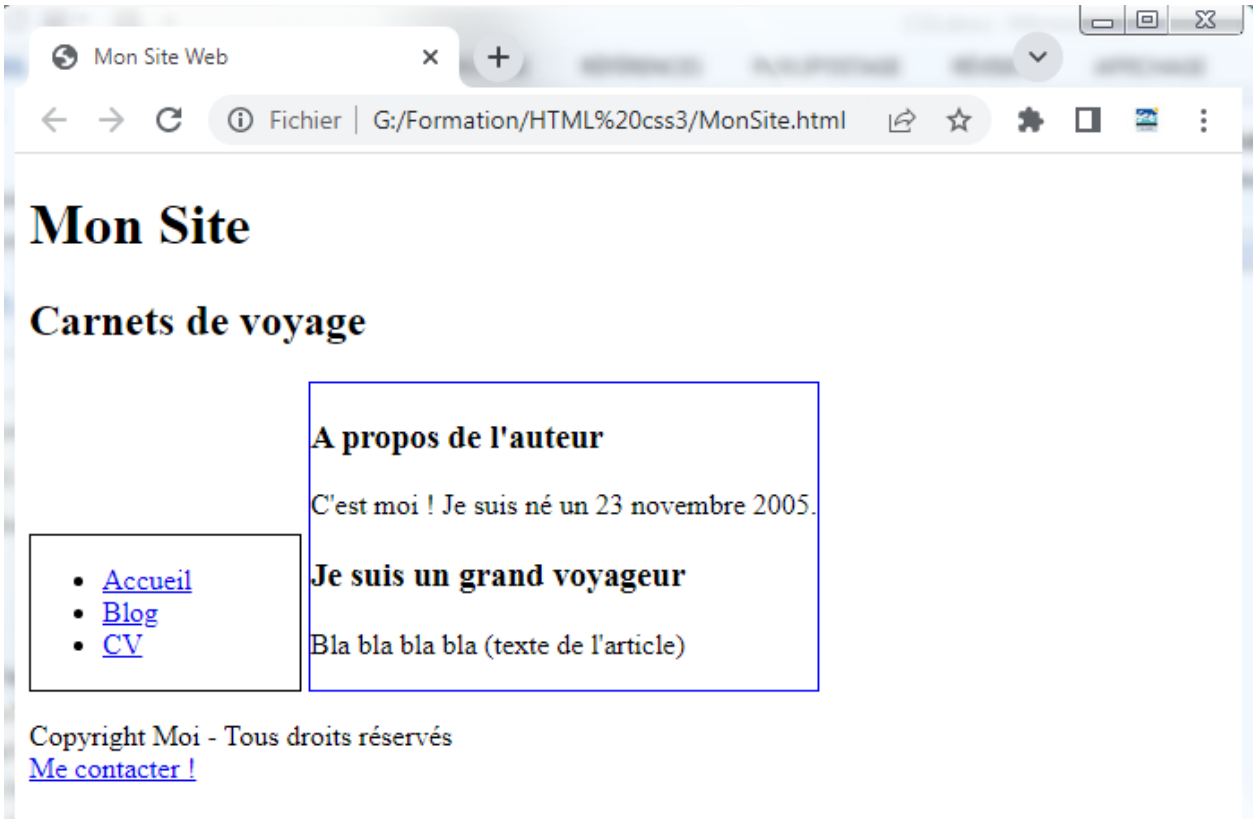
}

section
{
    display: inline-block;

    border: 1px solid blue;

}
```

Ce qui nous donne comme résultat la figure suivante.



Heureusement, le fait d'avoir transformé les éléments en inline-block nous permet d'utiliser une nouvelle propriété, normalement réservée aux tableaux : `vertical-align`. Cette propriété permet de modifier l'alignement vertical des éléments. Voici quelques-unes des valeurs possibles pour cette propriété :

- `baseline` : aligne de la base de l'élément avec celle de l'élément parent (par défaut) ;
- `top` : aligne en haut ;
- `middle` : centre verticalement ;
- `bottom` : aligne en bas ;
- (valeur en px ou %) : aligne à une certaine distance de la ligne de base (baseline).

```
nav
{
    display: inline-block;

    width: 150px;

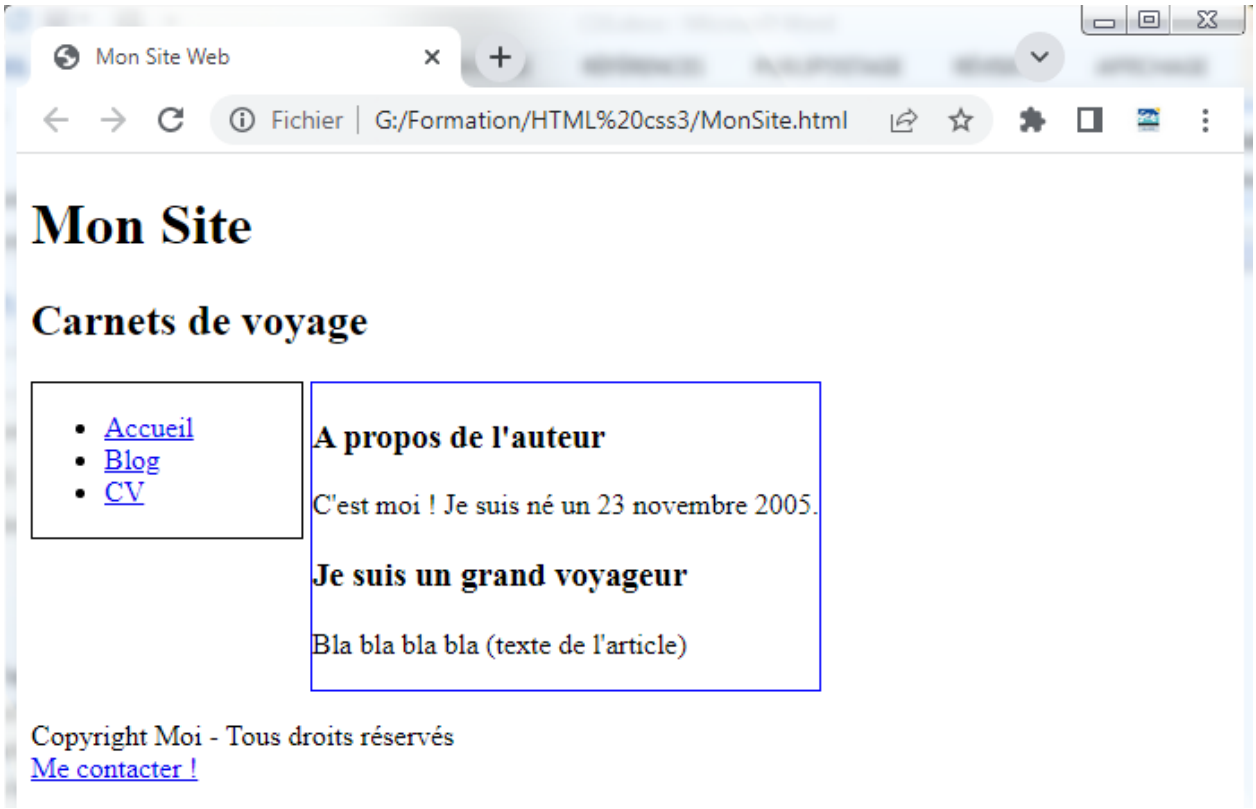
    border: 1px solid black;

    vertical-align: top;

}

section
{
    display: inline-block;
    border: 1px solid blue;
    vertical-align: top;

}
```



Les positionnements absolu, fixe et relatif

Il existe d'autres techniques un peu particulières permettant de positionner avec précision des éléments sur la page :

- **Le positionnement absolu** : il nous permet de placer un élément n'importe où sur la page (en haut à gauche, en bas à droite, tout au centre, etc.).
- **Le positionnement fixe** : identique au positionnement absolu mais, cette fois, l'élément reste toujours visible, même si on descend plus bas dans la page. C'est un peu le même principe que `background-attachment: fixed;` (si vous vous en souvenez encore).
- **Le positionnement relatif** : permet de décaler l'élément par rapport à sa position normale.

> Comme pour les flottants, les positionnements absolu, fixé et relatif fonctionnent aussi sur des balises de type inline.
> Toutefois, vous verrez qu'on l'utilise bien plus souvent sur des balises block que sur des balises inline.

Il faut d'abord faire son choix entre les trois modes de positionnement disponibles. Pour cela, on utilise la propriété CSS `position` à laquelle on donne une de ces valeurs :

- `absolute` : positionnement absolu ;
- `fixed` : positionnement fixe ;
- `relative` : positionnement relatif.

Nous allons étudier un à un chacun de ces positionnements.

Le positionnement absolu

Le positionnement absolu permet de placer un élément (réellement) n'importe où sur la page. Pour effectuer un positionnement absolu, on doit écrire :

```
element
{
    position: absolute;

}
```

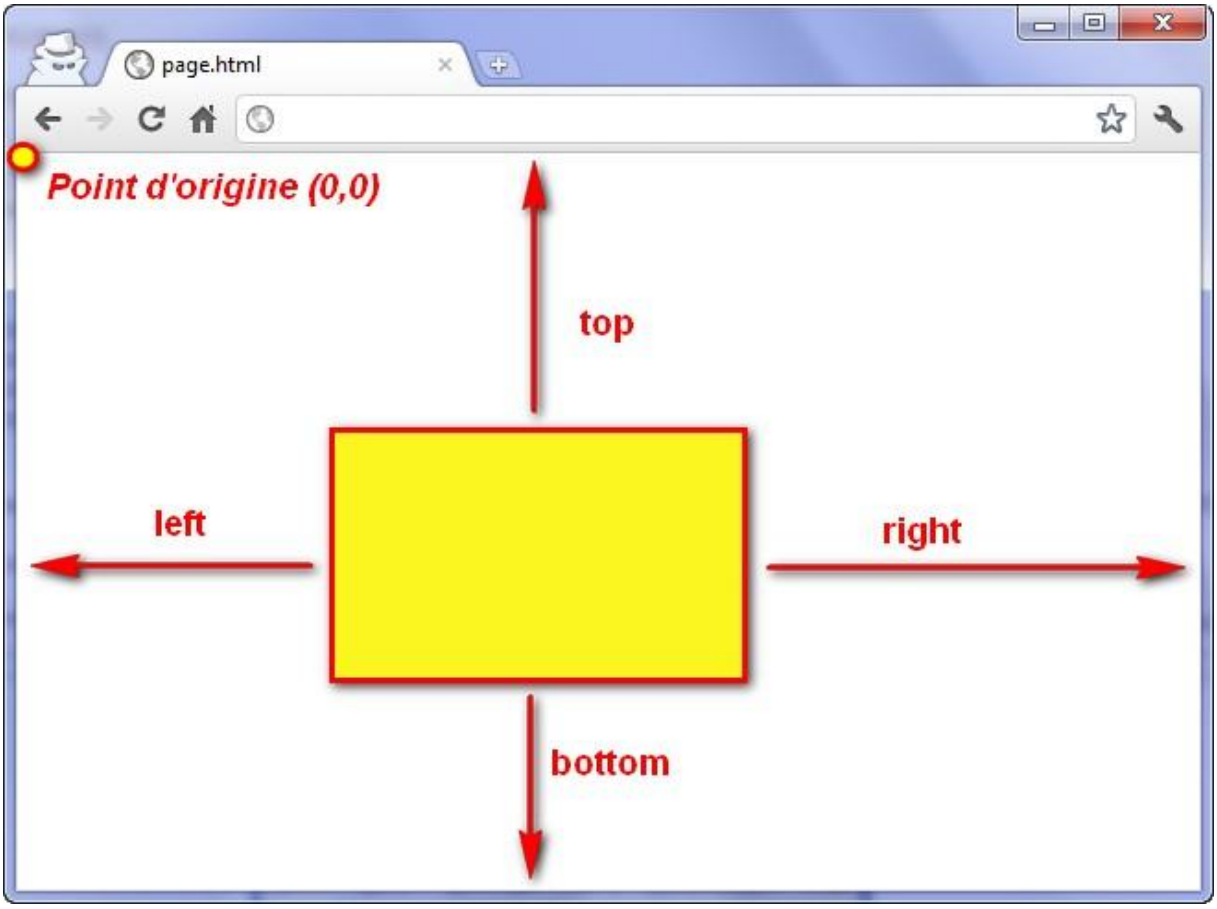
Mais cela ne suffit pas ! On a dit qu'on voulait un positionnement absolu, mais encore faut-il dire *où l'on veut que le bloc soit positionné sur la page*.

Pour ce faire, on va utiliser quatre propriétés CSS :

- `left` : position par rapport à la gauche de la page ;
- `right` : position par rapport à la droite de la page ;
- `top` : position par rapport au haut de la page ;
- `bottom` : position par rapport au bas de la page.

On peut leur donner une valeur en pixels, comme 14px, ou bien une valeur en pourcentage, comme 50%.

Si ce n'est pas très clair pour certains d'entre vous, la figure suivante devrait vous aider à comprendre.



Positionnement absolu de l'élément sur la page

Avec cela, vous devriez être capables de positionner correctement votre bloc.

Il faut donc utiliser la propriété `position` et au moins une des quatre propriétés ci-dessus (`top`, `left`, `right` ou `bottom`). Si on écrit par exemple :

```
element
{
    position: absolute;

    right: 0px;

    bottom: 0px;

}
```

… cela signifie que le bloc doit être positionné tout en bas à droite (0 pixel par rapport à la droite de la page, 0 par rapport au bas de la page).

On peut bien entendu ajouter une marge intérieure (padding) au menu pour qu'il soit moins collé à sa bordure.

Les éléments positionnés en absolu sont placés par-dessus le reste des éléments de la page ! Par ailleurs, si vous placez deux éléments en absolu vers le même endroit, ils risquent de se chevaucher. Dans ce cas, utilisez la propriété `z-index` pour indiquer quel élément doit apparaître au-dessus des autres.

```
element
{
    position: absolute;

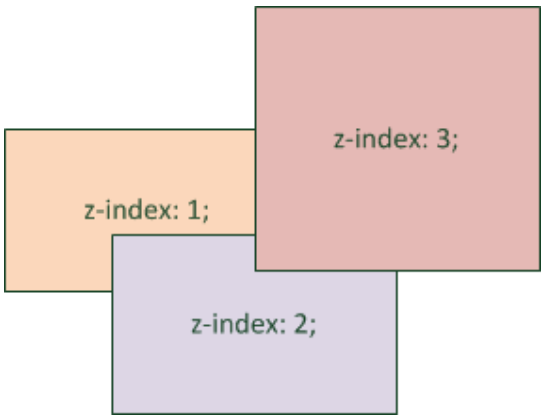
    right: 0px;
    bottom: 0px;
    z-index: 1;

}
element2
{
    position: absolute;
    right: 30px;
    bottom: 30px;

    z-index: 2;

}
```

L'élément ayant la valeur de `z-index` la plus élevée sera placé par dessus les autres, comme le montre la figure suivante.



Positionnement des éléments absolus

Le positionnement fixe

Le principe est *exactement le même* que pour le positionnement absolu sauf que, cette fois, le bloc reste fixe à sa position, même si on descend plus bas dans la page.

```
element
{
    position: fixed;
    right: 0px;
    bottom: 0px;

}
```

Essayez d'observer le résultat, vous verrez que le menu reste dans le cas présent affiché en bas à droite même si on descend plus bas dans la page (figure suivante).

Le positionnement relatif

Plus délicat, le positionnement relatif peut vite devenir difficile à utiliser. Ce positionnement permet d'effectuer des « ajustements » : l'élément est décalé par rapport à sa position initiale.

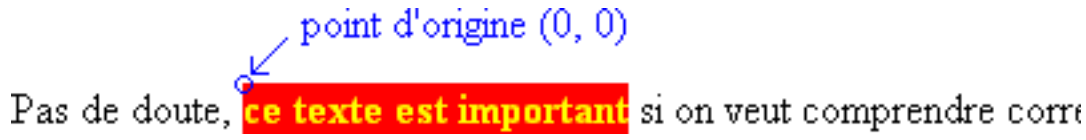
Prenons par exemple un texte important, situé entre deux balises `<strong>`. Pour commencer, je le mets sur fond rouge pour qu'on puisse mieux le repérer :

```
strong
{
   background-color: red; /* Fond rouge */

   color: yellow; /* Texte de couleur jaune */
```

Cette fois, le schéma que je vous ai montré tout à l'heure pour les positions absolue et fixe ne marche plus. Pourquoi ? Parce que l'origine a changé : le point de coordonnées (0, 0) ne se trouve plus en haut à gauche de votre fenêtre comme c'était le cas tout à l'heure. Non, cette fois l'origine se trouve en haut à gauche… de la position actuelle de votre élément.

Tordu n'est-ce pas ? C'est le principe de la position relative. Le schéma en figure suivante devrait vous aider à comprendre où se trouve l'origine des points.



Origine de la position relative

Donc, si vous faites un `position: relative;` et que vous appliquez une des propriétés `top`, `left`, `right` ou `bottom`, le texte sur fond rouge va se déplacer par rapport à la position où il se trouve.

Prenons un exemple : je veux que mon texte se décale de 55 pixels vers la droite et de 10 pixels vers le bas. Je vais donc demander à ce qu'il soit décalé de 55 pixels par rapport au « bord gauche » et de 10 pixels par rapportau « bord haut » (lignes 6 à 8) :

```
strong
{
   background-color: red;
   color: yellow;

   position: relative;
   left: 55px;
```

Le texte est alors décalé par rapport à sa position initiale, comme illustré à la figure suivante.



Le texte est décalé